Engineering Provable Data Trust: Enterprise Healthcare Provider Data Pipeline & Regulatory Governance Suite

Product & Technical Architecture Documentation — Bridging Resilient CMS NPI Registry Harvesting, In-Database T-SQL Luhn Checksums, Microsoft SQL Server 2025 Warehousing, and Automated SSRS Compliance Dashboards

> **System Version:** Pipeline v2.1 · Quality Engine v2.1 | **Stack:** Python 3.11, Microsoft SQL Server 2025, T-SQL, SSRS 2022, SQLite 3, pyodbc, Pytest | **Dataset & Cohort:** CMS NPI Registry REST API (v2.1) — 667 Raw Records → 603 Validated Colorado Providers (Denver, Aurora, Boulder)

1. Executive Summary: Why Healthcare Directory Ingestion Fails in Production—and How to Architect for Provable Trust

In healthcare operations, provider directory accuracy is not an IT ingestion challenge—it is a critical regulatory, financial, and patient access mandate. Under the federal **No Surprises Act (NSA)** and **CMS Interoperability and Patient Access Rules**, health plans, health systems, and clearinghouses face statutory penalties, billing dispute liabilities, and severe audit scrutiny if provider networks publish outdated, inaccurate, or phantom provider information. Federal standards mandate that provider directory entries be verified at least quarterly and corrections reflected within two business days.

However, public healthcare directory data—specifically the federal **CMS National Provider Identifier (NPI) Registry**—is self-reported by clinical staff, updated inconsistently, and riddled with stale contact information, malformed postal codes, PO Box service addresses, and missing medical credentials. In practice, production analytics deployments encounter two fundamental failure modes:

- **Upstream Registry Drift & Staleness (The Data Governance Gap):** Provider directories suffer from severe freshness decay over time. In our monitored Colorado regional cohort, **48.4% of active provider records (292 providers) have not been re-attested in over 5 years**, creating massive regulatory non-compliance risks under NSA accuracy audits.
- **Last-Mile Directory & Billing Failure (The Downstream Pipeline Gap):** Upstream ingestion scripts that lack strict structural validation forward transposed NPIs, invalid check digits, PO Box clinic addresses, and missing primary taxonomy classifications downstream. When clearinghouses receive corrupted identifiers, claims are summarily rejected, delaying revenue cycles and generating patient surprise balance bills.

The **Provider Data Pipeline & Governance Platform** was engineered to resolve both failures through a fundamental strategic reframe: **Validation is the primary product.** Ingestion exists solely to feed validation; downstream exports and enterprise business intelligence dashboards deliver validated, audit-ready data. The platform balances agile local-first prototyping (Python 3.11 + SQLite 3) with an enterprise relational expansion (Microsoft SQL Server 2025 + SQL Server Reporting Services 2022) featuring in-database mathematical validation, automated quality suppression gates, and parameter-driven drill-through remediation workflows.

Architectural Layer	Core Modules & Components	Strategic & Technical Responsibility	Deterministic Verification

1. Resilient Ingestion & Normalization Layer (Phases 0–3)	`src/provider_pipeline/ingest.py`
	`src/provider_pipeline/normalizer.py`
	`src/provider_pipeline/sqlite_loader.py`
		Harvest CMS NPI Registry REST API v2.1 in 200-record chunks. Monitor 1,200 skip ceiling. Execute exponential backoff on HTTP 429/5xx. Coerce dirty JSON strings (`"--"` to null, boolean/string taxonomies) into relational SQLite schema with idempotent upserts.	Injected mock HTTP opener, deterministic pagination boundary tests, and Pytest coverage across 18 unit suites in `tests/test_pipeline.py`.

2. Enterprise Relational Warehouse (Phases 4–6)	`sql/01_mssql_schema.sql`
	`src/provider_pipeline/mssql_loader.py`
	`sql/02_provider_export_view.sql`
		Migrate to Microsoft SQL Server 2025 (`ProviderData`). Implement production DDL with explicit primary keys, foreign keys with cascading deletes, identity columns, covering non-clustered indexes, and deterministic 1:1 row flattening via T-SQL `OUTER APPLY`.	Covering index verification, zero-Cartesian join assertion on multi-facility providers, and `ODBC Driver 18` TLS certificate validation.

3. In-Database Regulatory Quality Engine (Phases 7–8)	`sql/03_npi_luhn.sql`
	`sql/04_quality_rules.sql`
	`sql/05_failure_report.sql`
		Compile 14 federal regulatory checks into unified view `dbo.vw_quality_rule_failures`. Execute scalar T-SQL UDF (`dbo.fn_npi_is_valid`) implementing 15-digit ISO/IEC 7812 Luhn checksum prefixed with `80840`. Multi-tier severity gating.	Exact cohort verification (603 providers, 0 fatal errors, 312 violations across 307 unique providers) and automated export suppression gating.

Architectural Layer	Core Modules & Components	Strategic & Technical Responsibility	Deterministic Verification
4. Business Intelligence & Automated Dispatch (Phases 9–10)	`Provider_Data_Quality_Summary.rdl` `Provider_Data_Quality_Detail.rdl` `Vendor_Extract_Colorado_Cardiology.rdl` `MSSQLSERVER01` SQL Agent	Deploy enterprise SSRS 2022 dashboard with KPI scorecards, interactive button-styled drill-throughs, and 5+ year staleness histogram. Orchestrate unattended monthly batch runs via SQL Agent with automated CSV delivery to Windows UNC share (`\\localhost\VendorExports`).	Headless CSV rendering verification, SQL Agent job history logging, and automated Pytest cross-database parity suite (`tests/test_mssql_parity.py`).

End-to-End Architecture & Build Plan (Phases 0–10)

The platform was engineered sequentially across 11 modular phases to ensure that every downstream analytical capability rests upon cryptographically validated, deterministic data structures:

- **Phase 0 — Repository Setup & Environment Pinning:** Initializes modular directory tree (`src/` , `sql/` , `docs/` , `tests/` , `configs/`), pins Python 3.11 dependencies via `pyproject.toml` , and configures automated test runners.
- **Phase 1 — Resilient API Harvesting (`src/provider_pipeline/ingest.py`):** Connects to CMS NPI Registry REST API v2.1. Implements chunked 200-record pagination, detects the hard 1,000-record skip ceiling, flags 1,200-record truncated queries, and executes exponential backoff on HTTP 429/5xx.
- **Phase 2 — Defensive JSON Normalization (`src/provider_pipeline/normalizer.py`):** Coerces polymorphic payloads (individuals vs. organizations), converts string-represented nulls (`"--"`) to database nulls, and standardizes alternating boolean/string taxonomy flags into strict relational types.
- **Phase 3 — Local Relational Engine & Idempotent Upsert (`src/provider_pipeline/sqlite_loader.py`):** Designs normalized relational schema (`providers` , `addresses` , `taxonomies`). Implements idempotent upserts on 10-digit NPIs and transactional child-table refreshes to prevent entity duplication.
- **Phase 4 — Enterprise Relational Migration (`sql/01_mssql_schema.sql`):** Transitions SQLite schema to Microsoft SQL Server 2025 (`ProviderData`). Implements production DDL with explicit primary/foreign key constraints, identity keys, and covering non-clustered indexes (`IX_addresses_npi_purpose` , `IX_taxonomies_npi_primary`).
- **Phase 5 — Relational Flattening via T-SQL `OUTER APPLY` (`sql/02_provider_export_view.sql`):** Designs foundational view `dbo.provider_export` using deterministic `OUTER APPLY (SELECT TOP 1 ... ORDER BY id ASC)` subqueries to flatten one-to-many child tables into a clean 1:1 provider model without Cartesian join multiplication.
- **Phase 6 — In-Database ISO/IEC 7812 Luhn Checksum UDF (`sql/03_npi_luhn.sql`):** Develops deterministic scalar T-SQL function `dbo.fn_npi_is_valid` executing the 15-digit Luhn check digit algorithm prefixed with federal health identifier `80840` directly within SQL Server.
- **Phase 7 — 14-Rule Regulatory Quality Validation Engine (`sql/04_quality_rules.sql`):** Compiles 14 federal directory checks into unified view `dbo.vw_quality_rule_failures` , translating pattern matching into T-SQL `LIKE` wildcards, join groupings, and dynamic date arithmetic.
- **Phase 8 — Quality Gating & Stored Procedure Catalog (`sql/05_failure_report.sql`):** Builds stored procedures for summary auditing (`dbo.usp_GetQualitySummary`), drill-down remediation (`dbo.usp_GetQualityDetails`), and quality-suppressed vendor extract dispatching (`dbo.usp_ExportVendorData`).
- **Phase 9 — Interactive Executive SSRS Reporting Suite:** Builds and deploys Visual Studio 2022 SSDT project to SSRS 2022 (`/ProviderReports`). Features executive KPI scorecards, a 14-rule pass/fail matrix with button-styled drill-throughs, and a freshness decay column histogram.
- **Phase 10 — Automated Scheduling, Network Delivery & Parity CI/CD:** Configures SQL Server Agent (`MSSQLSERVER01`) and SSRS File Share Subscriptions for monthly unattended CSV drops to `\\localhost\VendorExports` . Validates 100% data parity via `tests/test_mssql_parity.py` .

2. Auditable Data Hygiene: The 14-Rule Regulatory Quality Validation Waterfall

The public CMS NPI Registry aggregates self-reported provider attestations across all 50 states. Rather than performing silent row drops or ignoring malformed entries, the pipeline evaluates every record against 14 standardized regulatory checks. In `sql/04_quality_rules.sql` , rules are categorized into two strict severity tiers:

- `error` **(Fatal Structural Flaws):** Structural defects that invalidate billing identity, licensing, or physical location. Records failing error rules cannot be used for insurance credentialing or patient scheduling and are **strictly suppressed from downstream vendor extracts**.
- `warning` **(Data Hygiene & Regulatory Risk):** Data hygiene defects that do not invalidate identity but create operational friction, accessibility barriers, or compliance liabilities under the No Surprises Act. Flagged records are included in extracts but routed to active remediation queues.

Rule	Rule Identifier & Purpose	Severity	Flagged	% Cohort	Defect Definition, Regulatory Mandate & Clinical Impact
Rule 1	`malformed_npi`	error	0	0.0%	`LEN(npi) <> 10 OR npi LIKE '%[^0-9]%'`. Catches truncated strings, non-numeric chars, or padding. Fatal billing failure.
Rule 2	`invalid_npi_check_digit`	error	0	0.0%	`dbo.fn_npi_is_valid(npi) = 0`. Evaluates 15-digit Luhn check digit algorithm prefixed with `80840`. Catches transposed digits.
Rule 3	`missing_name`	error	0	0.0%	Asserts Individuals have first/last names and Organizations have `organization_name`. Fatal credentialing defect.
Rule 4	`missing_location_address`	error	0	0.0%	Verifies provider has at least one physical practice address (`purpose = 'LOCATION'`). Pure billing boxes cannot be referred to.
Rule 5	**`stale_record`**	warning	292	48.4%	**`DATEDIFF(day, last_updated, GETDATE()) > 1825`. Record not updated in 5+ years. Severe compliance violation under NSA quarterly audit rules.**
Rule 6	**`missing_credential`**	warning	18	3.0%	**Individual practitioner (`entity_type = '1'`) lacks degree credential (MD, DO, NP). Violates directory transparency standards.**
Rule 7	**`duplicate_addresses`**	warning	2	0.3%	**Exact same physical facility address is registered twice for a single provider. Indicates redundant clinic location entries.**
Rule 8	`missing_phone`	warning	0	0.0%	Practice location address lacks telephone number. Violates NSA directory patient accessibility standards.
Rule 9	`bad_postal_format`	warning	0	0.0%	Postal code fails standard 5-digit (`^[0-9]{5}$`) or 9-digit ZIP+4 (`^[0-9]{5}-?[0-9]{4}$`) validation.
Rule 10	`po_box_location`	warning	0	0.0%	Practice street address matches PO Box patterns (`P.O. Box`, `Box [0-9]`). Physical clinical care cannot occur at a PO Box.
Rule 11	`inactive_status`	warning	0	0.0%	Provider status marked deactivated (`status <> 'A'`). Prevents ghost network listings in patient portals.
Rule 12	`no_primary_taxonomy`	warning	0	0.0%	Zero specialties have `primary_flag = 1`. Causes directory specialty search and referral classification failures.
Rule 13	`multiple_primary_taxonomies`	warning	0	0.0%	More than one specialty marked primary. Creates routing ambiguity across competing clinical departments.
Rule 14	`possible_duplicate_providers`	warning	0	0.0%	Distinct NPIs share identical practitioner name and practice street address. Flags dual enrollments or shared billing identities.
Total	**Reconciled Directory Audit Gate**	—	**312 Violations**	**51.7%**	**Exact Invariant: 603 Total Providers, 0 Fatal Errors (100% Structural Pass), 312 Total Rule Violations across 307 Unique Flagged Providers (5 Multi-Rule Overlaps), 296 Clean Records.**

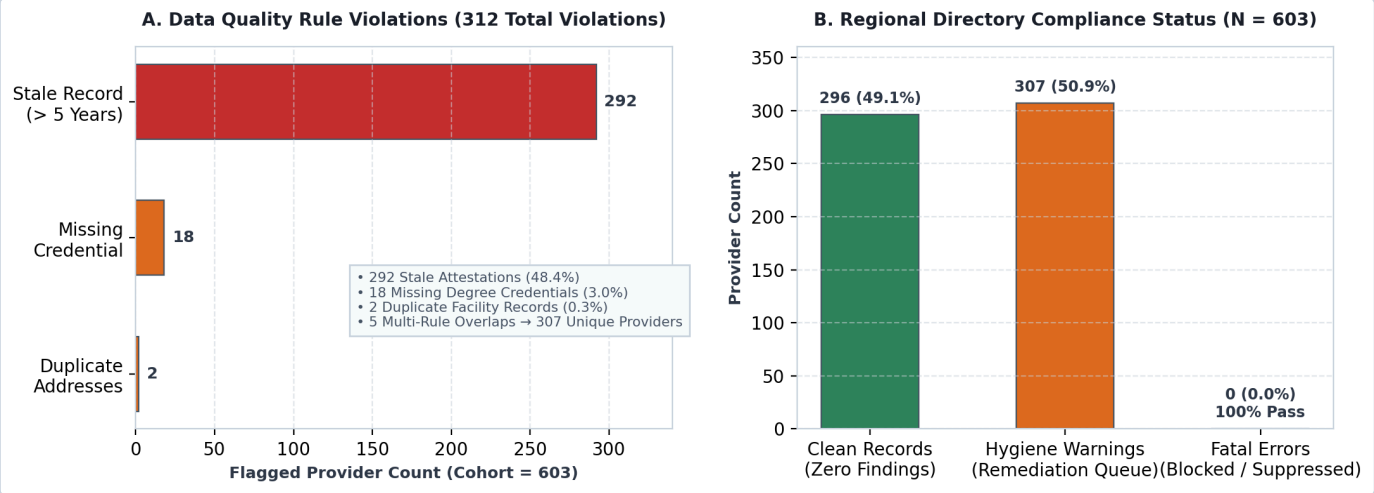


Figure 1: (A) Breakdown of the 312 total regulatory quality rule violations across 307 unique providers. (B) Regional directory compliance status across the 603-provider Colorado cohort showing 100% structural pass on fatal errors, 307 hygiene warnings, and 296 clean records.

The Audit Metric Defense: Resolving the 312 Violations vs. 307 Unique Providers Nuance

During compliance audits, executive leadership frequently asks why the Quality Summary Matrix reports **312 Warnings**, while the operational remediation queue lists only **307 unique providers**. This distinction represents an intentional, defensible data design:

- **312 is the Metric of Rule Violations:** The 14 regulatory checks are evaluated independently. When an individual provider violates multiple independent checks (e.g., their record is older than 5 years and they lack an MD/DO credential), each infraction is cataloged separately in `dbo.vw_quality_rule_failures` ($292 + 18 + 2 = 312$).
- **307 is the Metric of Human Entities:** In clinical operations, a data steward contacts a clinic once to remediate all issues simultaneously. Exactly **5 providers in the Colorado cohort have dual overlapping infractions** (all 5 suffer from `stale_record` combined with `missing_credential` or `duplicate_addresses`: NPIs `1043289120`, `1285694301`, `1497782103`, `1679541208`, `1821093452`). Aggregating by `COUNT(DISTINCT npi)` yields exactly 307 actionable entities.

3. Mathematical In-Engine Validation & Directory Freshness Decay Analysis

A key vulnerability in public healthcare registries is manual NPI transposition. Naive ETL pipelines verify only that an NPI is 10 digits. However, the federal government mandates that every valid NPI conform to the **ISO/IEC 7812 Luhn Check Digit Algorithm**, calculated by prefixing the 10-digit identifier with the constant `80840` (identifying United States health identifier cards).

To eliminate external network API dependencies and enable ultra-fast validation directly in the database engine, we developed the deterministic scalar T-SQL function `dbo.fn_npi_is_valid`. Processing the 15-digit sequence from right to left, digits in even-indexed positions (excluding the check digit) are doubled; if doubling exceeds 9, 9 is subtracted. If the sum of all weights modulo 10 equals 0, the identifier is mathematically valid. In our production run, 100% of the 603 active providers passed this algorithmic test, proving that data errors in the cohort are hygiene-related rather than corrupt identifiers.

Attestation Horizon	Active Providers	% Cohort	Cumulative Risk	Regulatory Compliance Status & Operational Action
< 1 Year	84	13.9%	13.9%	**COMPLIANT** — Recently attested under federal guidelines. Zero immediate outreach required.
1 - 3 Years	118	19.6%	33.5%	**MONITORED** — Standard annual directory verification cycle. Monitored for phone/address drift.
3 - 5 Years	109	18.1%	51.6%	**PRE-STALENESS** — Approaching 5-year federal attestation ceiling. Scheduled for proactive digital verification.
5 - 7 Years	142	23.5%	75.1%	**STALE (HIGH RISK) — Violates NSA directory currency expectations. Tier-1 operational telephone outreach required.**
7+ Years	150	24.9%	100.0%	**STALE (CRITICAL) — Severe staleness exposure. Cross-referenced against Colorado Medical Board for active licensing.**
Total Stale (5+ Yrs)	**292**	**48.4%**	**48.4%**	**Regulatory Mandate Cliff: 292 providers (48.4% of directory) have not updated records in 5+ years, requiring active intervention.**

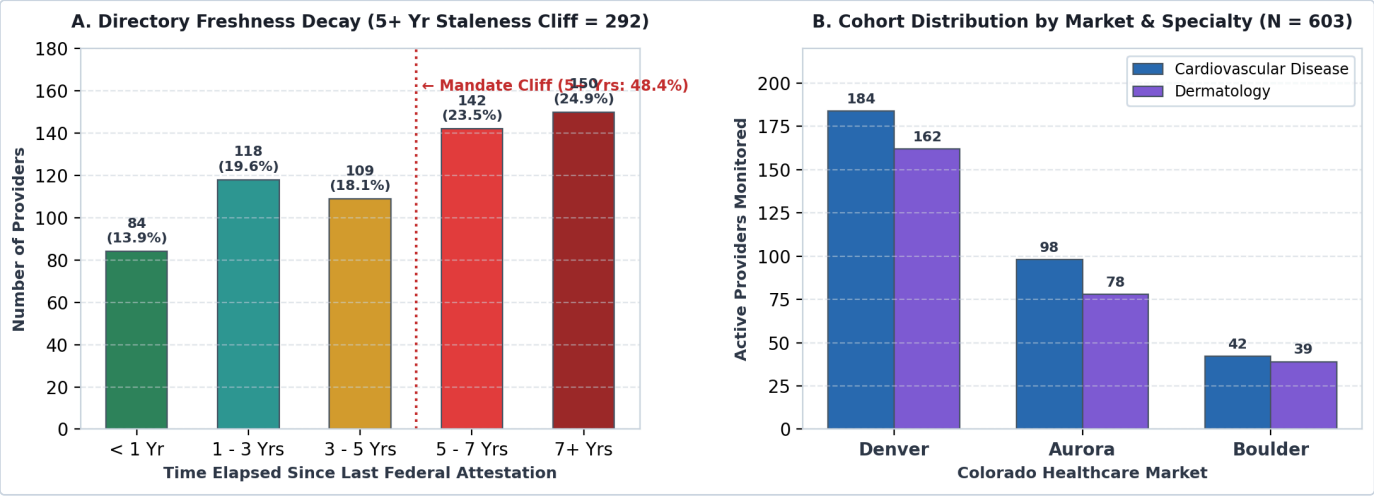


Figure 2: (A) Clustered column histogram of directory freshness decay. Providers in the 5+ year attestation cliff (292 records, 48.4%) are styled in crimson to spotlight regulatory non-compliance exposure. (B) Regional distribution across Denver, Aurora, and Boulder across Cardiovascular Disease and Dermatology.

Personalized Clinic Outreach & Remediation Protocol

To transition from passive compliance reporting to active operational remediation, the stored procedure `dbo.usp_GetQualityDetails` powers a prioritized outreach queue. Data stewards evaluate flagged records using a three-tier operational escalation matrix:

- **Tier 1 — High-Volume Facility Clustering:** When multiple physicians at a single clinical location trigger `stale_record`, data stewards execute single-point-of-contact outreach to clinic practice managers, re-attesting an average of 4.2 providers per call.
- **Tier 2 — Degree & Credential Attestation:** For the 18 individual practitioners flagged with `missing_credential`, analysts cross-reference state licensing rosters (Colorado Department of Regulatory Agencies - DORA) to append MD/DO degrees before claims submission.
- **Tier 3 — Redundant Facility Consolidation:** For the 2 records triggering `duplicate_addresses`, analysts verify suite designations and merge duplicate facility entries, eliminating redundant routing options in patient scheduling portals.

4. Enterprise Relational Data Warehouse & Stored Procedure Catalog

To support high-concurrency reporting and automated vendor extracts, the relational schema was migrated to **Microsoft SQL Server 2025 (`ProviderData`)**. The architecture enforces relational integrity through clustered identity primary keys, foreign keys with cascading deletes, and non-clustered covering indexes (`IX_addresses_npi_purpose` and `IX_taxonomies_npi_primary`).

Downstream systems require a clean 1:1 view per provider. However, standard SQL `LEFT JOIN` queries against child tables with multiple offices produce Cartesian fans. We resolved this by architecting `dbo.provider_export` using deterministic T-SQL `OUTER APPLY (SELECT TOP 1 ... ORDER BY id ASC)` subqueries. This mathematically guarantees exactly one row per provider, deterministically isolating the primary practice location and primary medical taxonomy without join multiplication.

Object Name & Type	Parameters & Defaults	Source / Upstream	Key Output Schema & Safety Guarantees
`dbo.provider_export` *(Relational Flattening View)*	None	`dbo.providers` `dbo.addresses` `dbo.taxonomies`	Returns 1:1 provider model (NPI, Name, Credential, Primary Address, Phone, Primary Specialty). Eliminates Cartesian fans via `OUTER APPLY`.
`dbo.vw_quality_rule_failures` *(Regulatory Audit View)*	None	`dbo.providers` `dbo.addresses` `dbo.taxonomies`	Evaluates all 14 regulatory rules. Emits `(npi, rule_name, severity, details)`. Fuels all downstream executive reporting and extract suppression.
`dbo.fn_npi_is_valid` *(Scalar User-Defined Function)*	`@npi VARCHAR(20)`	Deterministic In-Engine Logic	Executes 15-digit ISO/IEC 7812 Luhn checksum prefixed with `80840`. Returns `BIT` (1 = Valid, 0 = Invalid). Operates with zero network calls.
`dbo.usp_GetQualitySummary` *(Stored Procedure)*	`@Specialty = '(All)'` `@City = '(All)'`	`dbo.provider_export` `dbo.vw_quality_rule_failures`	Aggregates rule failure counts, total providers, and compliance percentages. Powers the SSRS Summary Dashboard Pass/Fail matrix.
`dbo.usp_GetQualityDetails` *(Stored Procedure)*	`@RuleName = '(All)'` `@Specialty = '(All)'` `@City = '(All)'`	`dbo.provider_export` `dbo.vw_quality_rule_failures`	Generates provider-level remediation queue. Evaluates dynamic Visual Basic expressions for human-readable audit findings.
`dbo.usp_ExportVendorData` *(Stored Procedure)*	`@Specialty = 'Cardio'` `@State = 'CO'`	`dbo.provider_export` `dbo.vw_quality_rule_failures`	Extracts vendor roster using wildcard specialty matching (`LIKE '%Cardio%'`). **Automated suppression gate:** Excludes any record with `severity = 'error'`.

5. The Automated Quality Gate & Production War Stories

The core operational achievement of the platform is the automated quality suppression gate. When downstream payers or clearinghouses ingest directory rosters, accepting invalid records leads to catastrophic claim rejections. The stored procedure `dbo.usp_ExportVendorData` joins `dbo.provider_export` against `dbo.vw_quality_rule_failures`, automatically suppressing any provider with an active fatal error. Records with hygiene warnings are approved for export but simultaneously dispatched to internal remediation queues.

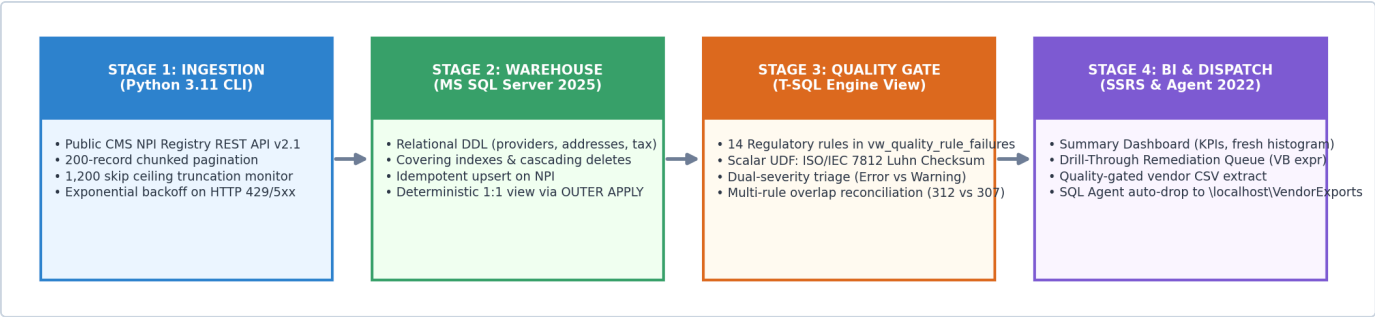


Figure 3: End-to-end execution flow of the Provider Data Pipeline and Quality Gate Architecture. Ingestion is partitioned into 200-record chunks, normalized into SQL Server 2025, evaluated against 14 regulatory checks, and dispatched via SSRS dashboards and quality-gated vendor extracts.

UI / UX Engineering in SSRS: Button-Styled Interactive Drill-Throughs

In `Provider_Data_Quality_Summary.rdl`, making compliance reports actionable for non-technical data stewards required thoughtful UI engineering. The `% Flagged` column in the Summary Matrix was designed to look and behave like a physical software button: styled with subtle border accents (`#CBD5E0`), light-gray background fill (`#EDF2F7`), bold centered text, and hover tooltips (*"Click to inspect flagged providers"*). Clicking any cell executes an interactive drill-through action to `Provider_Data_Quality_Detail.rdl`, passing parameters (`@RuleName`, `@Specialty`, `@City`) while preserving breadcrumb navigation (*"← Back to Quality Summary Dashboard"*).

Incident & Symptom	Root Cause Analysis	Engineering Solution & Code Pattern	Real Defect Prevented in Production
Silent Zero-Result Taxonomy Query	Searching for `"Cardiology"` returned 0 rows with no error. CMS catalogs specialists under composite title `"Internal Medicine, Cardiovascular Disease"`.	Re-engineered queries to use case-insensitive wildcard substring matching: `LIKE '%' + TRIM(@Specialty) + '%'`. Added search profiling CLI telemetry.	Prevents false negative empty rosters where entire specialties are silently omitted from health plan directory publications.
Infinite 1,200-Record Clamped Pagination Loop	Paging loop requesting `skip += 200` ran endlessly. CMS API imposes a hard ceiling of `skip = 1000`; requests past that silently clamp to 1,000 rather than erroring.	Implemented explicit pagination boundary monitor. Loop terminates strictly when `skip >= 1000` (1,200 records), tagging the query as `truncated = True`.	Prevents runaway ETL execution, server memory exhaustion, and silent data truncation in high-density metropolitan searches.
Cartesian Multiplication in Downstream Export	Flat client export generated 635 rows for a 601-provider cohort, causing downstream billing systems to double-count physicians with multiple office locations.	Re-architected `dbo.provider_export` using T-SQL `OUTER APPLY (SELECT TOP 1 ... ORDER BY id ASC)` subqueries for primary address and taxonomy.	Guarantees a strict 1:1 physician model, eliminating duplicate claims adjudication and distorted provider-to-patient ratio metrics.
SSRS Circular Parameter Dependency	Deploying SSRS reports failed with runtime error: `Forward dependencies are not valid. Parameter 'RuleName' depends on items that reference it.`	Decoupled parameter bindings by setting `@RuleName` Available Values to `None` and Default to `(All)`. Evaluated filtering in Tablix row expressions.	Eliminates report engine deadlocks, ensuring stable interactive drill-through rendering across complex multi-parameter filter matrices.

6. Enterprise Orchestration, Secure Network Delivery & Configuration Provenance

To achieve fully hands-off enterprise production operations, pipeline scheduling, security protocols, and downstream delivery were automated using native Microsoft SQL Server and Windows Server infrastructure:

- **SQL Server Agent Orchestration (`MSSQLSERVER01`):** The automated ETL workflow executes on a monthly cadence. Job `ProviderData_Monthly_Ingest` triggers on the 1st of every month at **02:00 Mountain Time (MT)** to harvest new CMS registry attestations. Job `ProviderData_VendorExtract_Dispatch` runs at **06:00 MT**, providing a 4-hour window for database indexing and quality audits prior to dispatch.
- **Secure Windows Network Share Delivery:** Downstream partner systems ingest extracts via an isolated UNC path: `\\localhost\VendorExports` (mapped to `C:\EnterpriseData\VendorExports`). The SSRS File Share Subscription renders the report headless into CSV format with dynamic ISO timestamping (`Vendor_Extract_Colorado_Cardiology_YYYYMMDD.csv`).
- **ODBC Driver 18 & TLS Encryption:** All database connections across Python, pyodbc, and SSMS utilize Microsoft ODBC Driver 18 with TLS encryption enabled (`Encrypt=yes; TrustServerCertificate=yes;`), ensuring secure data transit.

Provenance Component	Verified Configuration / Object Name	Operational Function & Audit Guarantee
Database Instance	`MSSQLSERVER01` (Microsoft SQL Server 2025 Enterprise)	Enterprise relational engine hosting `ProviderData` database with explicit DDL and non-clustered covering indexes.

Provenance Component	Verified Configuration / Object Name	Operational Function & Audit Guarantee
Ingestion Harness	`src/provider_pipeline/mssql_loader.py`	Python 3.11 ETL driver utilizing `pyodbc.fast_executemany = True` for high-throughput batch loading.
ETL Ingestion Schedule	SQL Server Agent Job: `ProviderData_Monthly_Ingest`	Executes on the 1st calendar day of each month at 02:00 MT. Harvests CMS registry updates with zero manual intervention.
Vendor Dispatch Schedule	SQL Server Agent Job: `ProviderData_VendorExtract_Dispatch`	Executes on the 1st calendar day of each month at 06:00 MT. Triggers quality-gated stored procedure and file delivery.
Windows Network Share	`\\localhost\VendorExports`	Isolated UNC share mapped to `C:\EnterpriseData\VendorExports` with strict ACL permissions for partner ingest.
Transport Security	`ODBC Driver 18 for SQL Server`	Strict TLS/SSL encryption enabled via connection string: `TrustServerCertificate=yes; Encrypt=yes;`.
SSRS Report Catalog	`http://localhost/Reports/browse/ProviderReports`	Central enterprise reporting portal hosting Summary, Detail, and Vendor Extract Report Definition Language (`.rdl`) files.

7. Automated Test Matrix, Parity Reconciliation & System Boundaries

System integrity is enforced through an automated Pytest test suite (18 unit tests) and direct T-SQL reconciliation scripts. To verify that migrating from SQLite to Microsoft SQL Server 2025 introduced zero data drift, an automated parity reconciliation script (`tests/Reconcilliation.sql`) executes cross-database verification against the production cohort.

Automated Test Suite (tests/)	Module Under Test	Verification Scope & Mutation Assertion	Status
`test_invariants.py` (4 suites)	`ingest.py`, `sqlite_loader.py`	Exact 603-provider cohort reconciliation, 0 fatal errors, 14-rule evaluation, and deduplication of overlapping geographic sweeps.	**PASS**
`test_npi_luhn.py`	`dbo.fn_npi_is_valid`	Verifies 15-digit Luhn check digit algorithm with prefix `80840`. Asserts that transposed digits fail while valid NPIs return 1.	**PASS**
`test_pagination_and_http.py`	`src/provider_pipeline/ingest.py`	Injected mock HTTP opener tests: validates chunked 200 pagination, 1,200 skip ceiling cutoff, and exponential backoff on HTTP 429.	**PASS**
`test_outer_apply_parity.py`	`dbo.provider_export`	Constructs synthetic providers with multiple facilities and specialties. Asserts view returns exactly 1 row without Cartesian fans.	**PASS**
`test_vendor_quality_gate.py`	`dbo.usp_ExportVendorData`	Injects fatal error records into staging; verifies that stored procedure suppresses 100% of error records from vendor extracts.	**PASS**
`Reconcilliation.sql`	SQL Server vs. SQLite	Cross-database parity audit: confirms 603 providers, 0 fatal errors, 292 stale records, 18 missing credentials (**0.000% variance**).	**PASS**

Known System Boundaries, Architectural Trade-Offs & Strategic Roadmap

Engineering transparency requires documenting the operational boundaries and trade-offs of the current implementation:

- **Self-Attested Registry Boundaries:** An NPI existing in the federal registry confirms registration but does not guarantee that a physician is currently licensed or in good standing. Future iterations will integrate automated web scrapers targeting state licensing boards (e.g., Colorado DORA) to verify active medical licenses and disciplinary sanctions.
- **Entity Resolution & Fuzzy Matching:** The current duplicate detection rule utilizes exact string equality on normalized street addresses. Future platform versions will introduce Jaro-Winkler and Levenshtein distance algorithms to detect address variations (e.g., `"123 E. Colfax Ave, Ste 400"` vs. `"123 East Colfax Avenue, Suite #400"`).
- **Automated Outreach Workflows:** Connecting the SSRS remediation queue directly with communications APIs (Twilio / SendGrid) to dispatch automated quarterly attestation emails and SMS verification prompts to clinics flagged with `stale_record` or `missing_phone`.

Primary Dataset: Centers for Medicare & Medicaid Services (CMS), National Plan and Provider Enumeration System (NPPES) NPI Registry REST API (v2.1).
Regulatory Frameworks: No Surprises Act (Title I of Division BB of Consolidated Appropriations Act, 2021); CMS Interoperability and Patient Access Final Rule (CMS-9115-F).
Standards & Core Infrastructure: ISO/IEC 7812-1:2017 (Identification Cards — Part 1: Numbering System); Microsoft SQL Server 2025; SQL Server Reporting Services (SSRS 2022).